

Problem #4

Part (i)

Using '\ Results:

```
A=[ -4  0  1  1;  
    0  0  1  0;  
    0 -2  4  3;  
    0 -1  0  4];  
b = [2 4 0 0]';  
xbuilt = A\b
```

```
xbuilt = 4x1  
1.3000000000000000  
12.800000000000001  
4.000000000000000  
3.200000000000000
```

Using gauss_jordan:

```
xnum=gauss_jordan(A,b)
```

```
xnum = 4x1  
1.3000000000000000  
12.800000000000001  
4.000000000000000  
3.200000000000000
```

Comparing results (Percent error):

```
for i=1:4  
    percenterror(i) = 100*abs((xnum(i)-xbuilt(i))/xbuilt(i));  
end  
percenterror'
```

```
ans = 4x1  
0  
0  
0  
0
```

Part(ii)

Calculating inverse using Gauss_Jordan:

```
Ainv=gauss_jordan_inv(A)
```

```
Ainv = 4x4  
-0.250000000000000  0.450000000000000 -0.050000000000000  0.100000000000000  
0 3.200000000000000 -0.800000000000000  0.600000000000000  
0 1.000000000000000 0 0  
0 0.800000000000000 -0.200000000000000  0.400000000000000
```

Verifying that the calculated inverse of A by using gauss_jordan is correct:

```
Ainv*A
```

```
ans = 4x4
    1     0     0     0
    0     1     0     0
    0     0     1     0
    0     0     0     1
```

```
Ainv*A == A*Ainv
```

```
ans = 4x4 logical array
    1     1     1     1
    1     1     1     1
    1     1     1     1
    1     1     1     1
```

Calculating inverse of A using built in function inv:

```
Ainvp = inv(A)
```

```
Ainvp = 4x4
   -0.250000000000000    0.450000000000000   -0.050000000000000    0.100000000000000
         0    3.200000000000000   -0.800000000000000    0.600000000000000
         0    1.000000000000000         0         0
         0    0.800000000000000   -0.200000000000000    0.400000000000000
```

Comparing results by a percent error matrix:

```
pem=zeros(4);
for i=1:4
    for j=1:4
        if Ainvp(i,j)==0
            pem(i,j)=0;
        else
            pem(i,j)=100*abs((Ainv(i,j)-Ainvp(i,j))/Ainvp(i,j));
        end
    end
end
pem
```

```
pem = 4x4
    0     0     0     0
    0     0     0     0
    0     0     0     0
    0     0     0     0
```

The percent error is 0 for every entry.

```
function [x] = gauss_jordan(A,b)
n=size(A,1);
AA=[A b];
for i=1:n
    if (AA(i,i) == 0)
        AA=pivot_row(AA,i);
    end
    AA(i,:)=AA(i,:)/AA(i,i);
    for j=1:n
        if (j ~= i)

```

```

        AA(j,:)=AA(j,:)-AA(j,i)*AA(i,:);
    end
end
end
x=AA(:,n+1);
end

function [A_inv] = gauss_jordan_inv(A)
n=size(A,1);
I=eye(n);
AA=[A I];
for i=1:n
    if (AA(i,i) == 0)
        AA=pivot_row(AA,i);
    end
    AA(i,:)=AA(i,:)/AA(i,i);
    for j=1:n
        if (j ~= i)
            AA(j,:)=AA(j,:)-AA(j,i)*AA(i,:);
        end
    end
end
A_inv=AA(:,n+1:2*n);
end

function [A] = pivot_row(A,i)
[value,index]=max(abs(A(:,i)));
row_temp=A(i,:);
A(i,:)=A(index,:);
A(index,:)=row_temp;
end

```